



Introduction to Neo4j



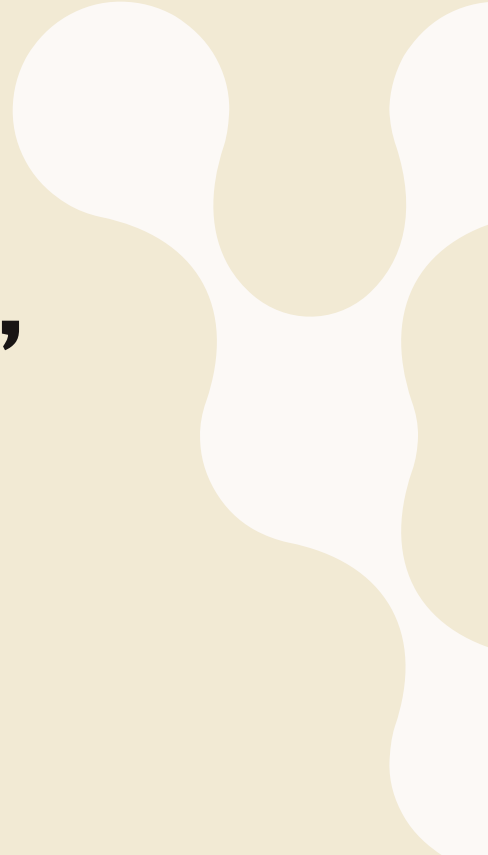
Martin

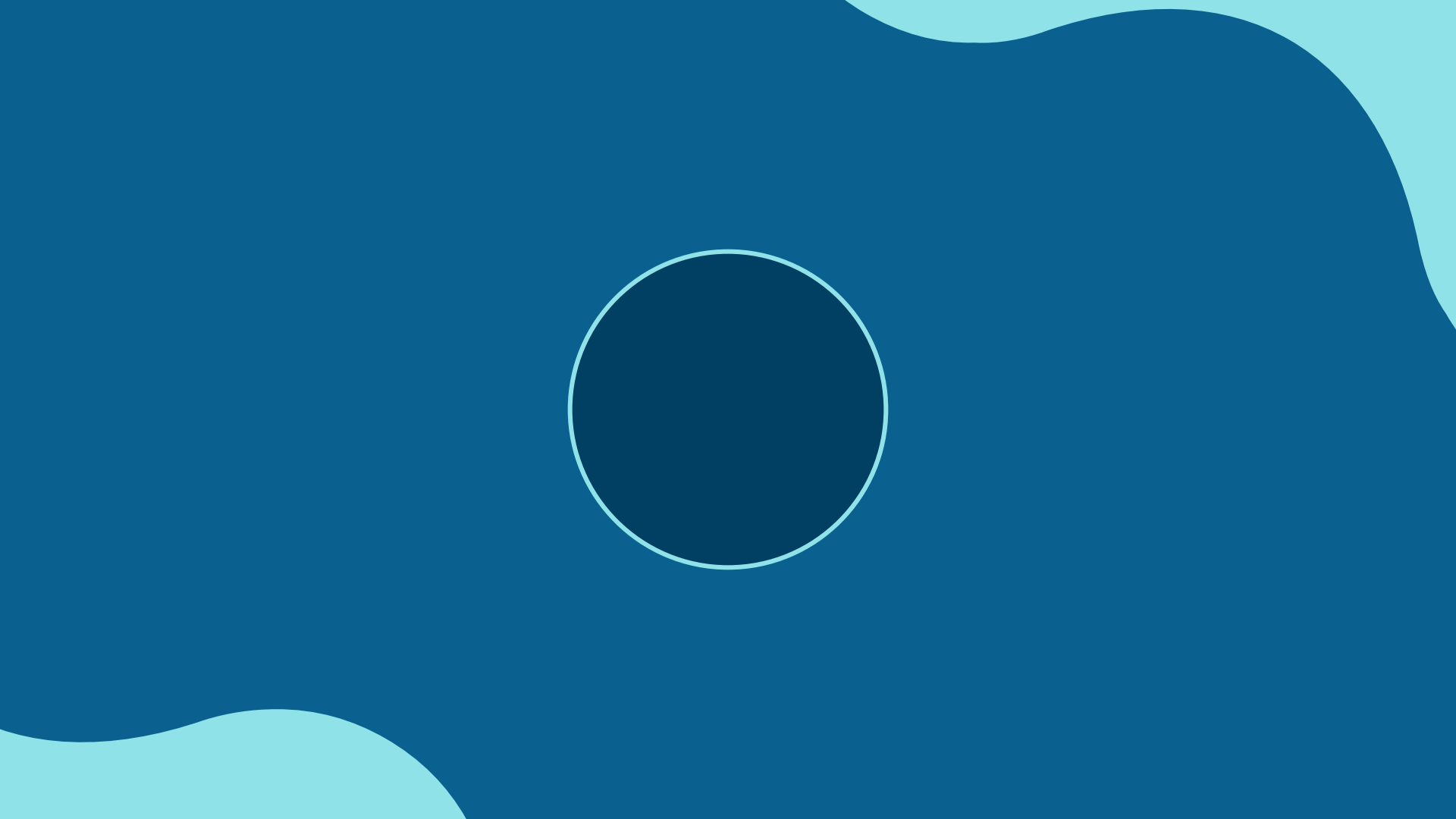
Today

- Concepts
- Data modeling
- Cypher -reading, writing, aggregation
- Vector indexes (if we have time)



Nodes, relationships, properties





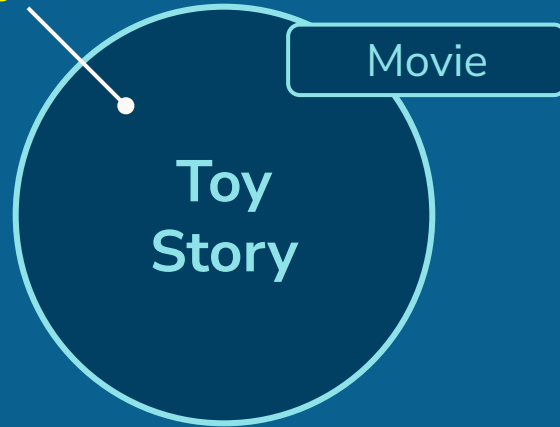


Toy
Story

Nodes represent *things*

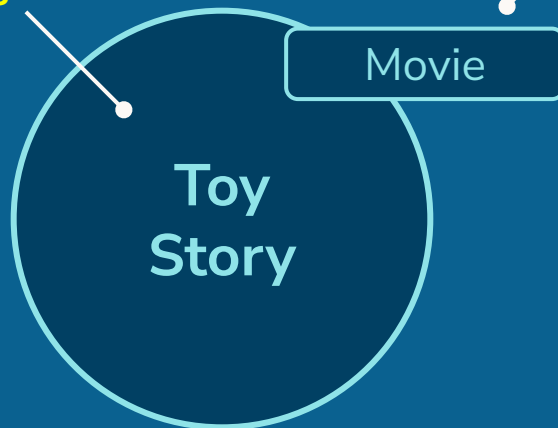


Nodes represent *things*

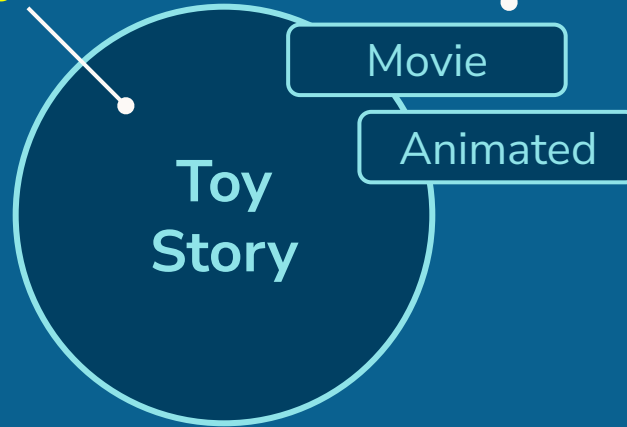


Nodes represent *things*

Nodes can be identified by
one or more *labels*



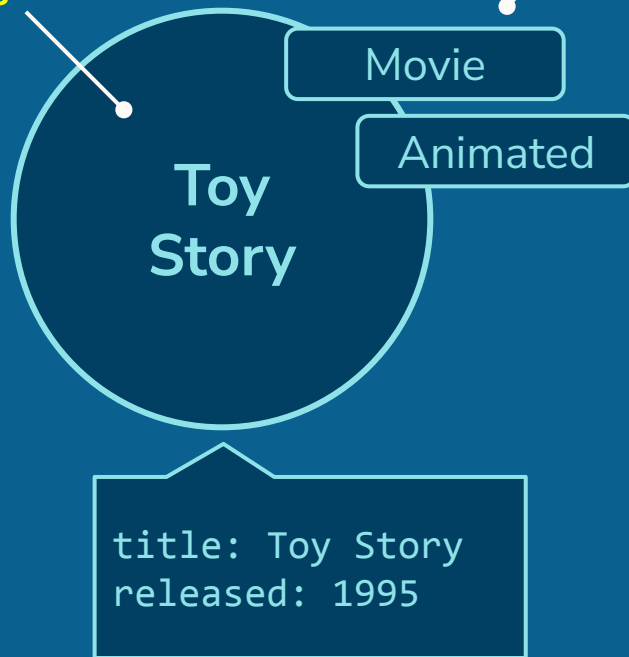
Nodes represent *things*



Nodes can be identified by one or more *labels*

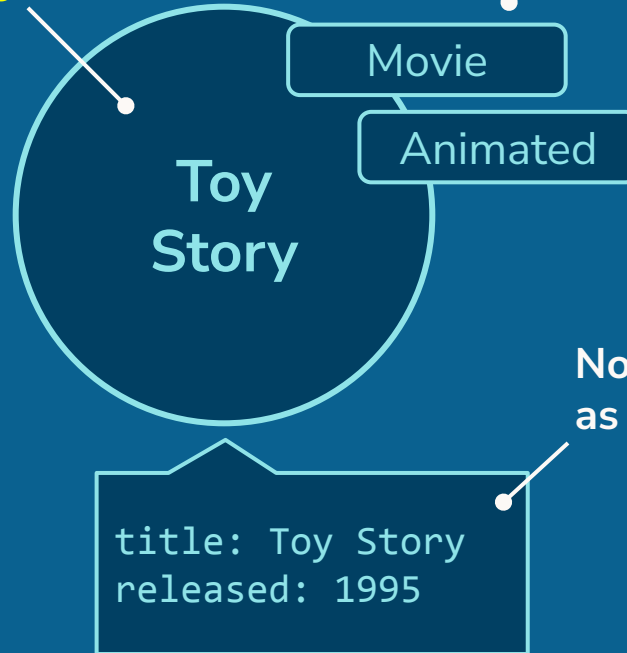
Nodes represent *things*

Nodes can be identified by
one or more *labels*



Nodes represent *things*

Nodes can be identified by one or more *labels*



Nodes can hold *properties* as key/value pairs

**Toy
Story**

Movie

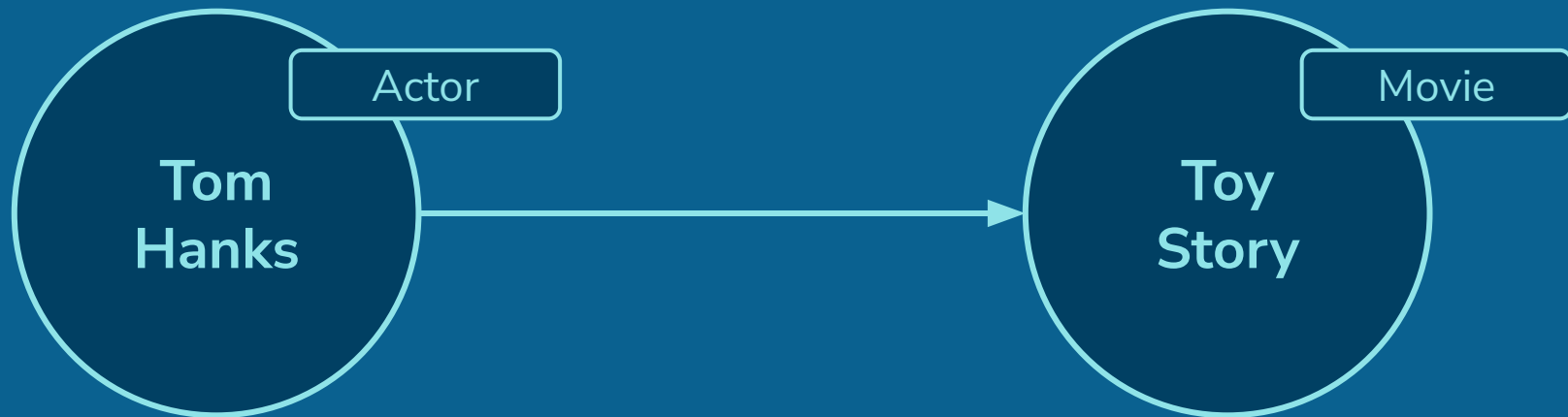
Actor

The diagram consists of two circular nodes on a dark blue background. The left node is labeled 'Tom Hanks' and has a rectangular label 'Actor' attached to its top right. The right node is labeled 'Toy Story' and has a rectangular label 'Movie' attached to its top right. The background features light blue wavy shapes at the top and bottom.

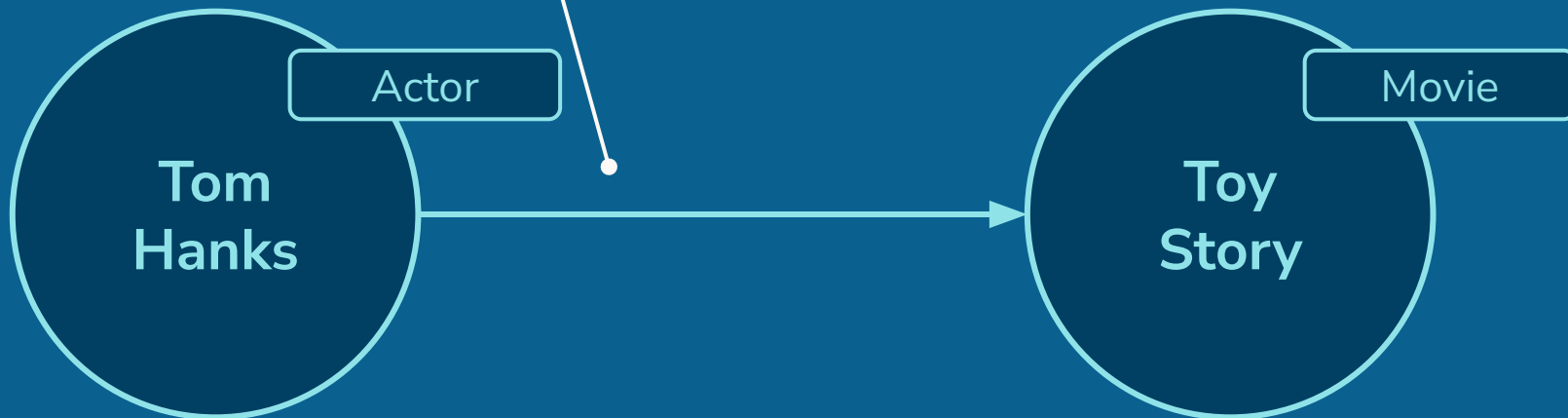
Tom
Hanks

Movie

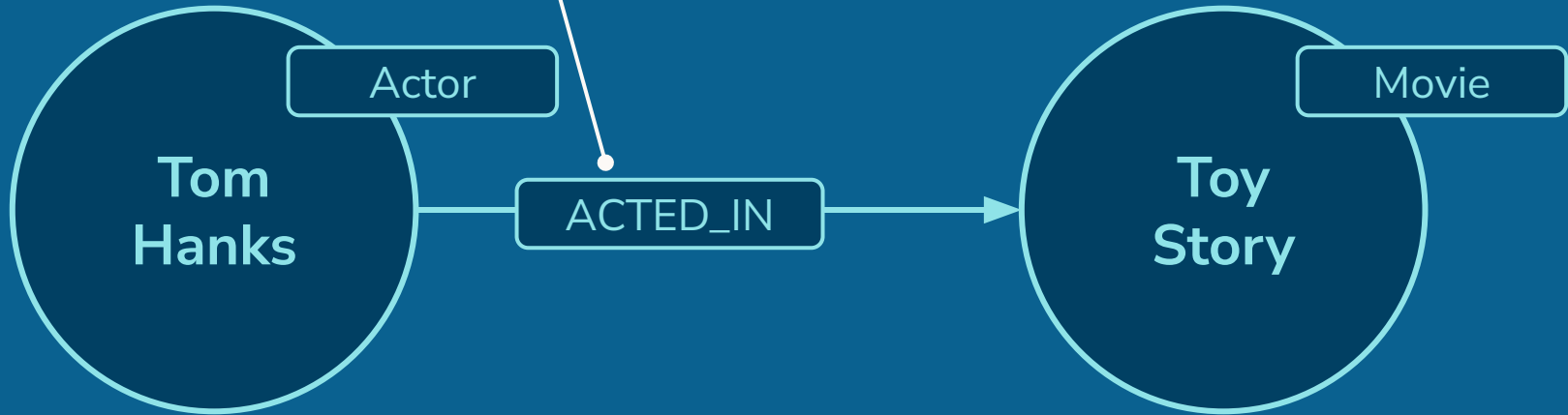
Toy
Story



Relationships connect *two nodes*

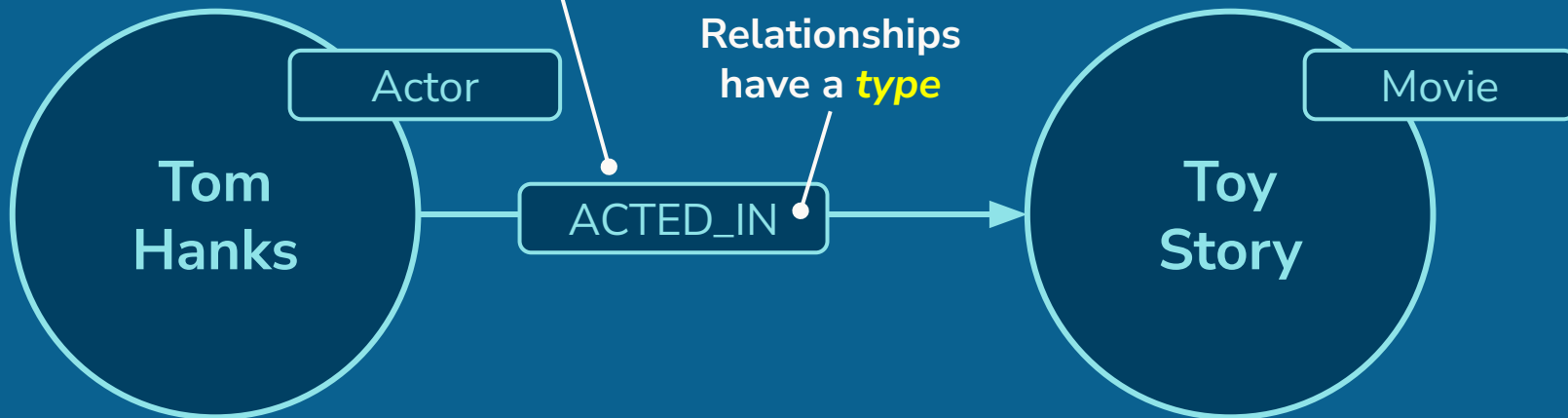


Relationships connect *two nodes*

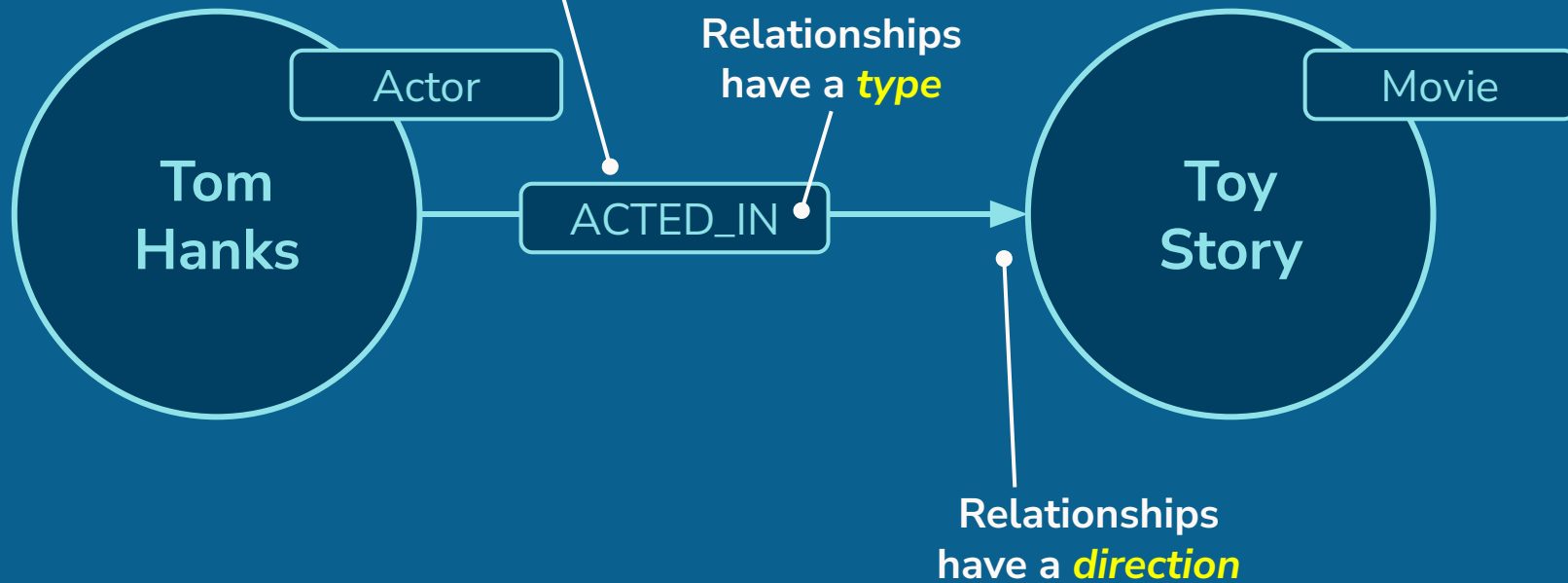


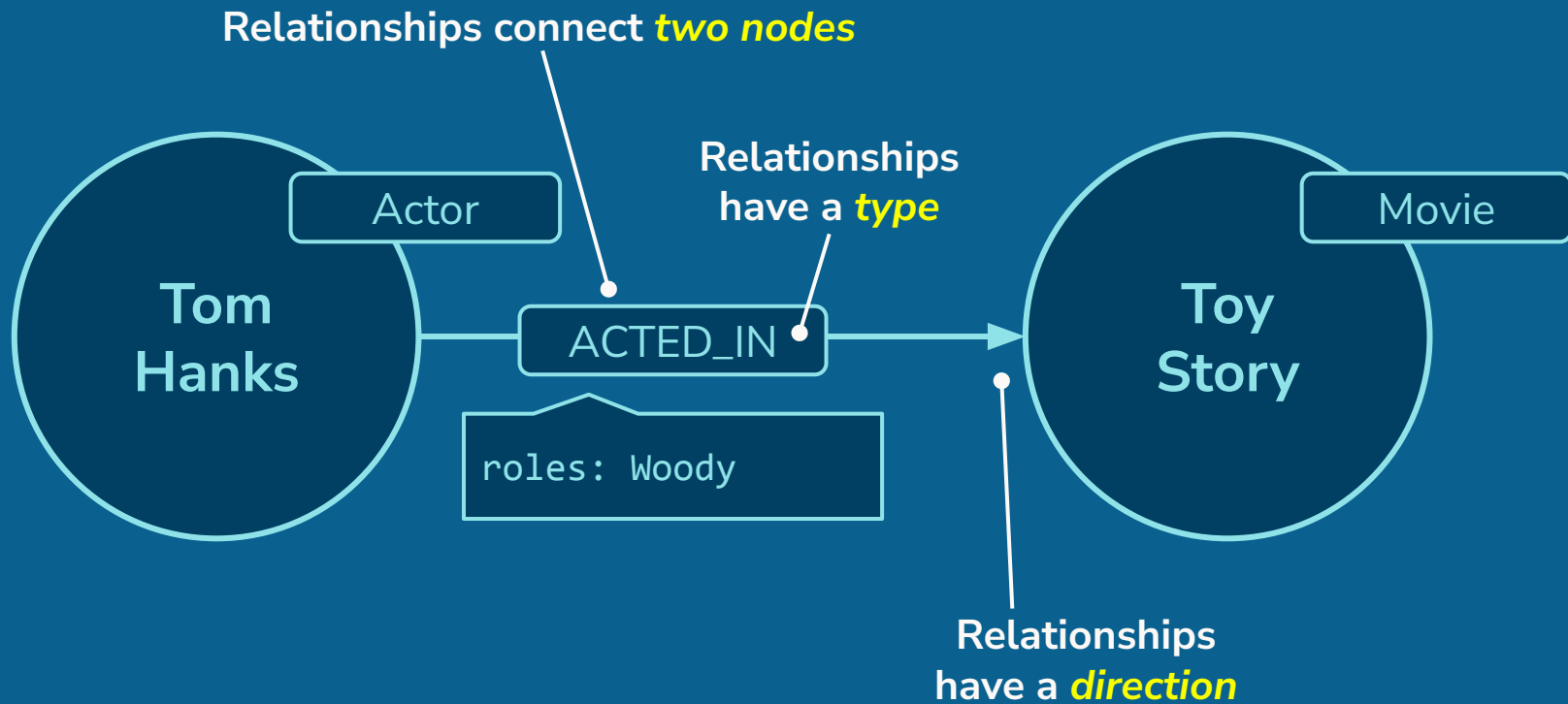
Relationships connect *two nodes*

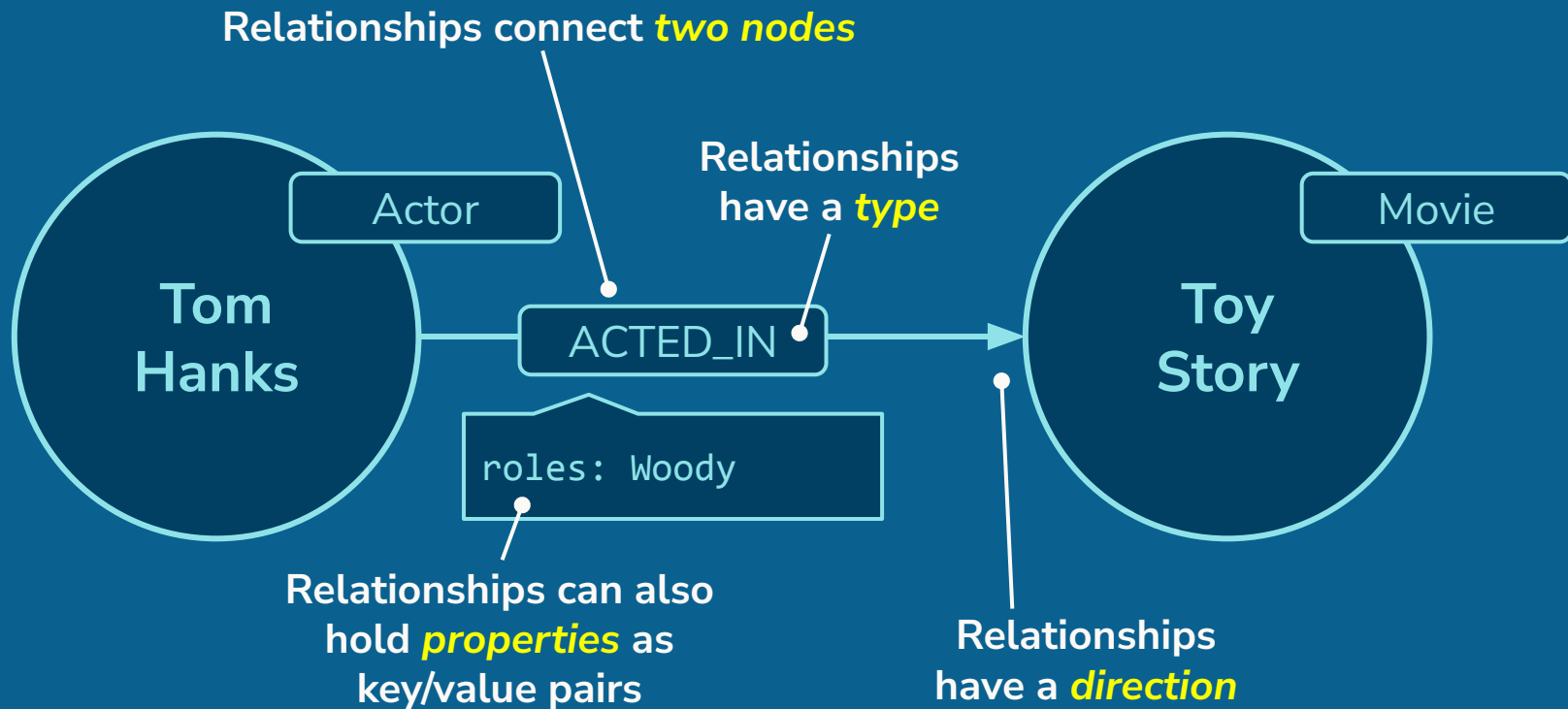
Relationships
have a *type*

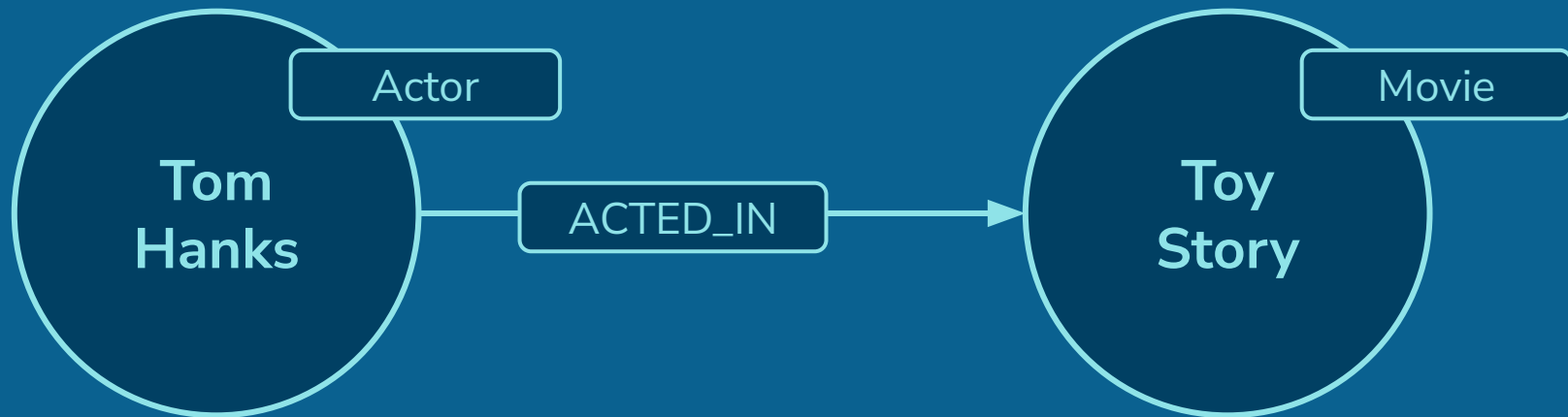


Relationships connect *two nodes*









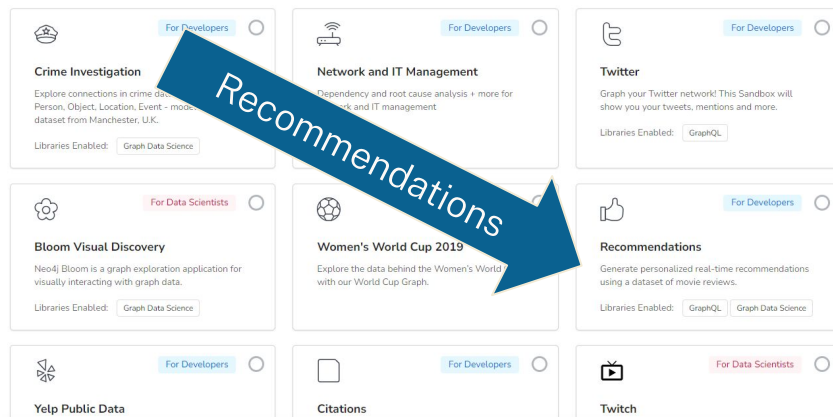
Let's explore!

sandbox.neo4j.com



Scroll-down to:

Pre-built data



Project : Recommendations



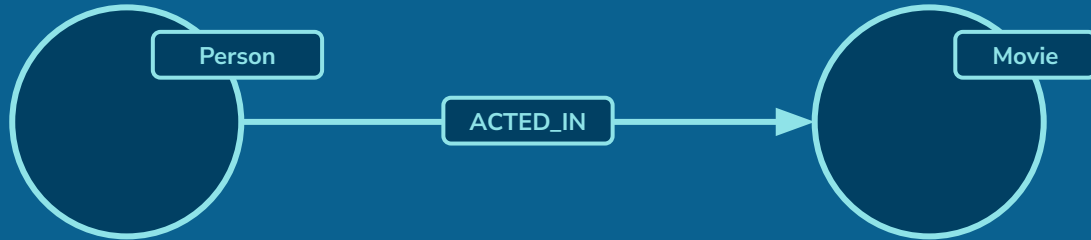
Create



Cypher



Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets, dashes and arrows**.

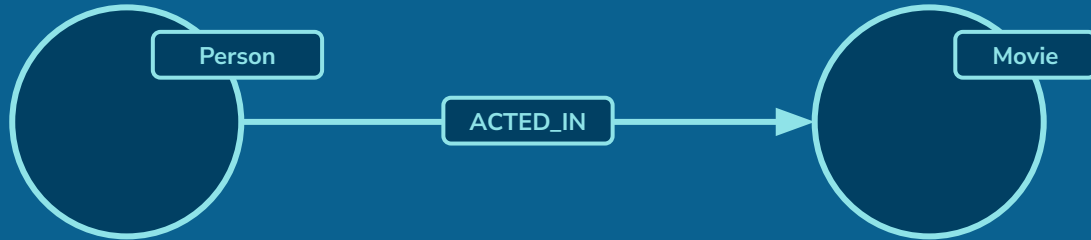


`(p:Person)-[r:ACTED_IN]->(m:Movie)`

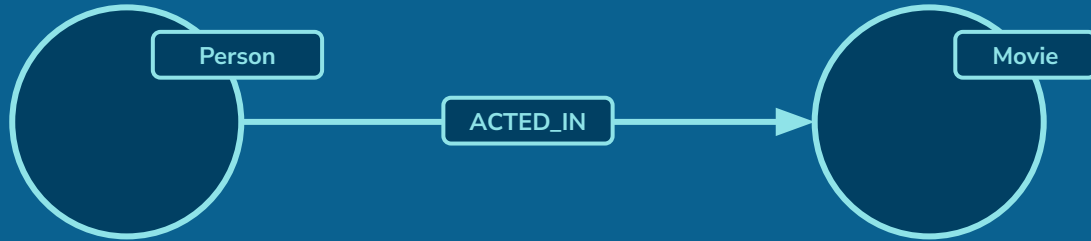
Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, **dashes** and **arrows**.

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, **dashes** and **arrows**.

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an ASCII-art style syntax consisting of **brackets**, **dashes** and **arrows**.

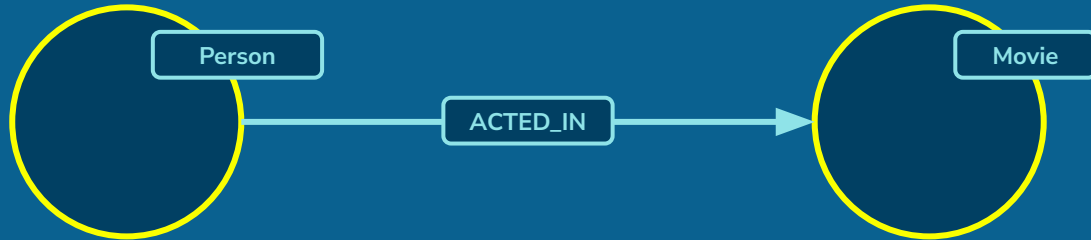


Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets, dashes and arrows**.



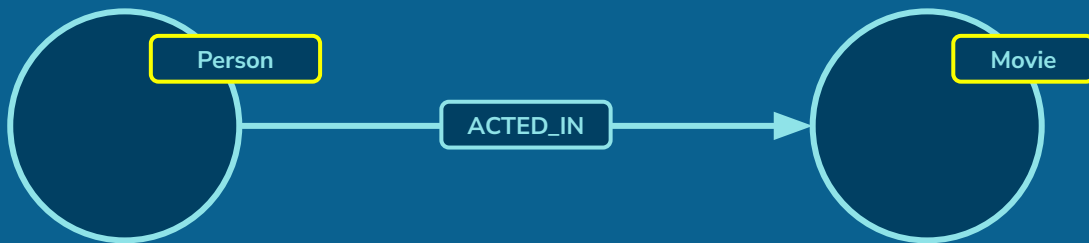
```
(p:Person)-[r:ACTED_IN]->(m:Movie)
```

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, dashes and arrows.



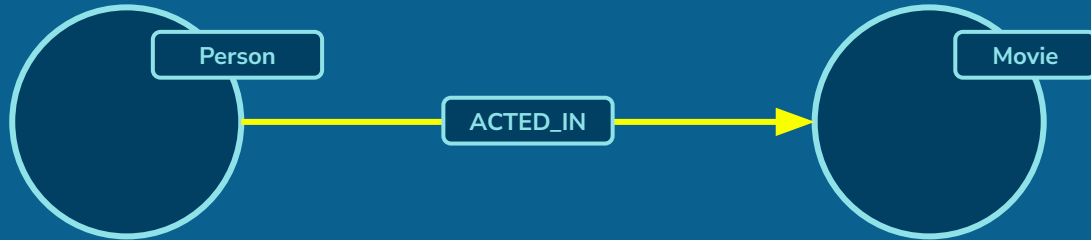
```
(p:Person)-[r:ACTED_IN]->(m:Movie)
```

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, dashes and arrows.



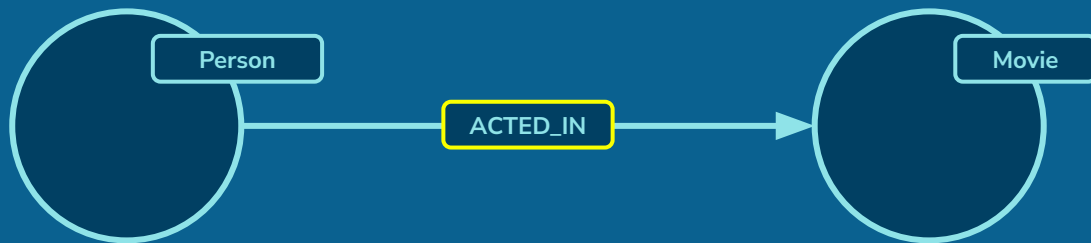
```
(p:Person)-[r:ACTED_IN]->(m:Movie)
```

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, **dashes** and arrows.



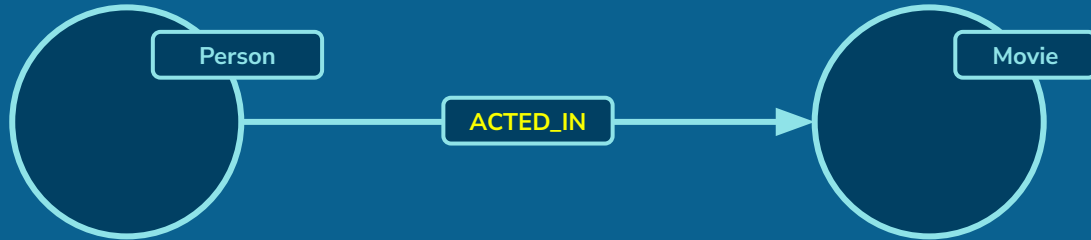
`(p:Person) - [r:ACTED_IN] -> (m:Movie)`

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, **dashes** and arrows.



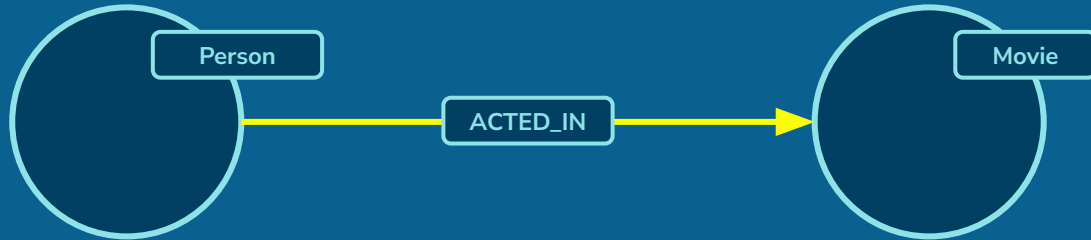
```
(p:Person)-[r:ACTED_IN]->(m:Movie)
```

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, **dashes** and arrows.



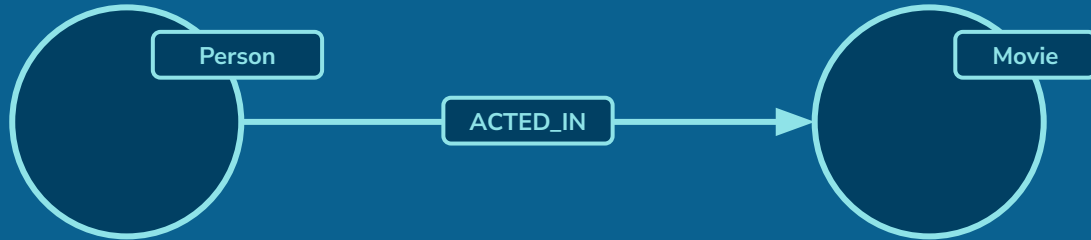
```
(p:Person)-[r:ACTED_IN]->(m:Movie)
```

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, **dashes** and **arrows**.



```
(p:Person)-[r:ACTED_IN]->(m:Movie)
```

Cypher is a **declarative** language that allows you to **identify patterns** in your data using an **ASCII-art style syntax** consisting of **brackets**, **dashes** and **arrows**.



```
(p:Person)-[r:ACTED_IN]->(m:Movie)
```



Reading



Reading from Neo4j

// Find a pattern in the database

MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)

Reading from Neo4j

// Find a pattern in the database

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
```

// Filter on a node property

```
WHERE m.title = "Toy Story"
```

Reading from Neo4j

```
// Find a pattern in the database
```

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
```

```
// Filter on a node property
```

```
WHERE m.title = "Toy Story"
```

```
// Choose what to return
```

```
RETURN p.name AS actor, r.roles AS roles
```


Reading from Neo4j

// Find a pattern in the database

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
```

SQL Equivalent

FROM/JOIN

// Filter on a node property

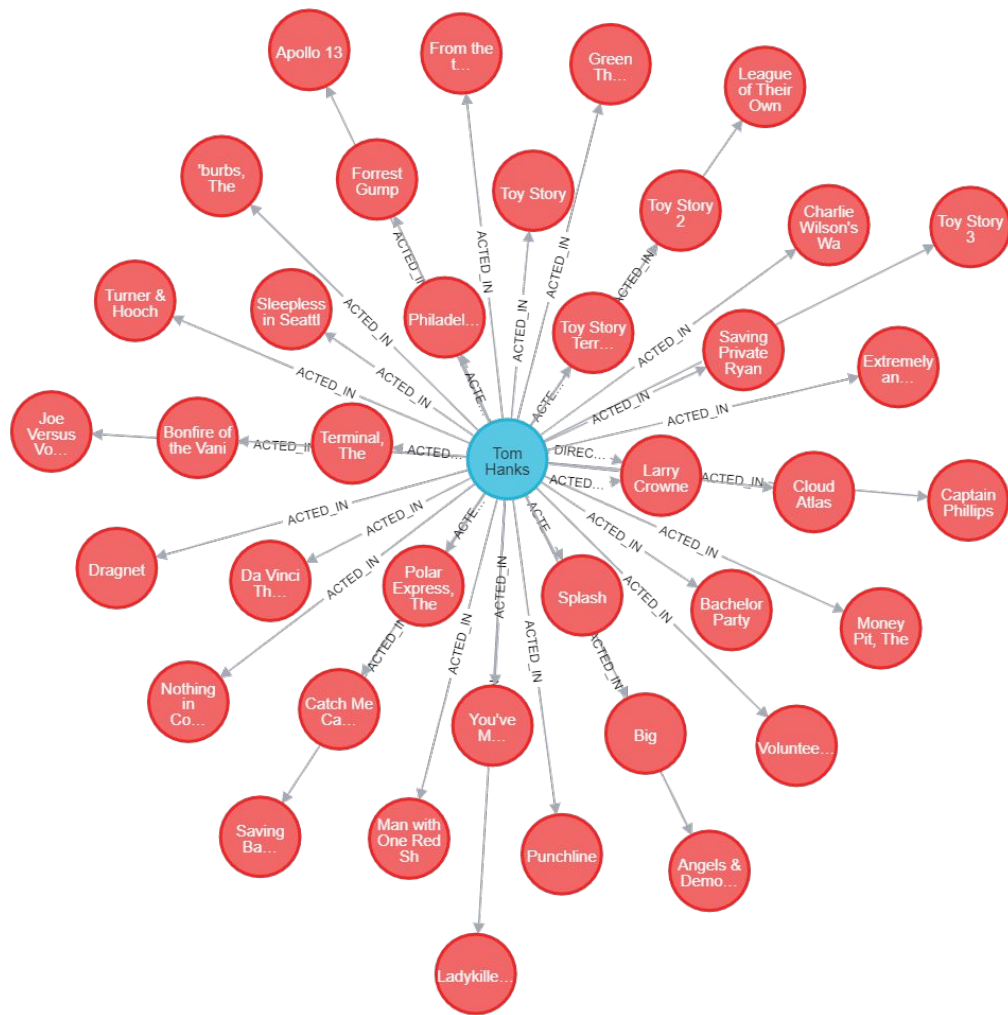
```
WHERE m.title = "Toy Story"
```

WHERE

// Choose what to return

```
RETURN p.name AS actor, r.roles AS roles
```

SELECT



Have a go!

1. Find a Movie you like.
`MATCH (m:Movie) RETURN m.title`
2. Use the ACTED_IN relationship to find who acted in the movie.
`MATCH (m:Movie {title: "???"})-[:???????]->(p:Person)
RETURN p.name`
3. Use the DIRECTED relationship to find who directed the movie
`MATCH (m:Movie {title: "???"})-[:???????]->(p:Person)
RETURN p.name`



Writing



Writing to Neo4j

// Find or create using MERGE

MERGE (p:Person {name: "Tom Hanks"})

Writing to Neo4j

// Find or create using MERGE

MERGE (p:Person {name: "Tom Hanks"})

MERGE (m:Movie {title: "Toy Story"})

Writing to Neo4j

// Find or create using MERGE

MERGE (p:Person {name: "Tom Hanks"})

MERGE (m:Movie {title: "Toy Story"})

MERGE (p)-[r:ACTED_IN]->(m)

Writing to Neo4j

// Find or create using MERGE

MERGE (p:Person {name: "Tom Hanks"})

MERGE (m:Movie {title: "Toy Story"})

MERGE (p)-[r:ACTED_IN]->(m)

SET r.roles = "Woody"



Have a go!

1. Create a Person node for yourself
`MERGE (me:Person{name: "???????"})`
2. Give yourself a role in a movie by creating an ACTED_IN relationship
`MATCH (me:Person{name: "???????"})
MATCH (m:Movie{title: "???????"})
MERGE (me)-[r:???????]->(m)
SET r.role = "???????"`
3. Find your role in the movie
`MATCH (p:Person{name:"???????"})-[r:ACTED_IN]->(?????)
RETURN ?.?????`

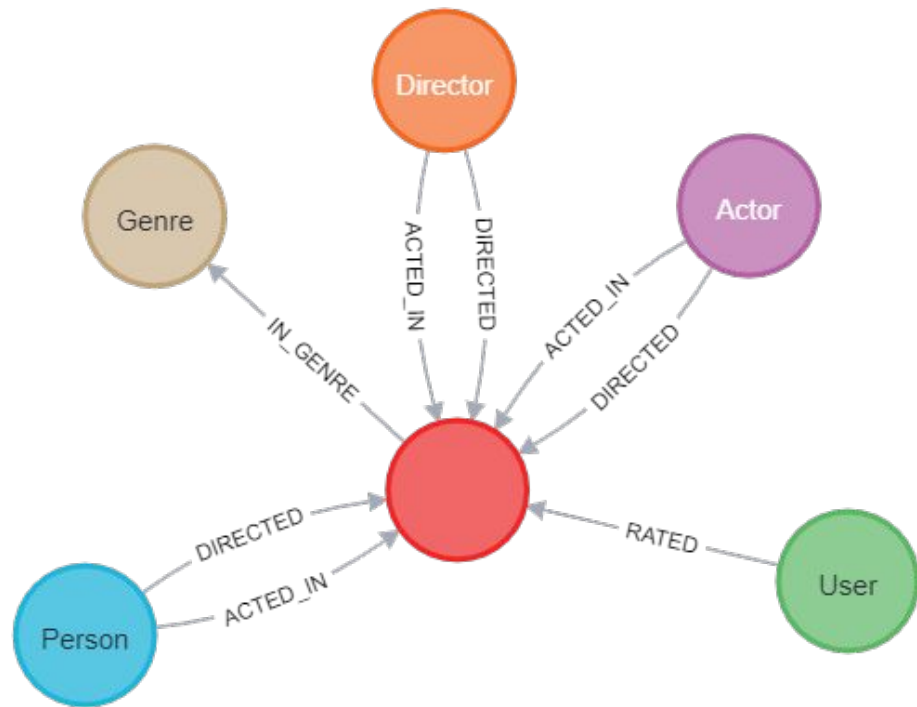


Data model



Take a look!

CALL `db.schema.visualization()`



The Golden Rule



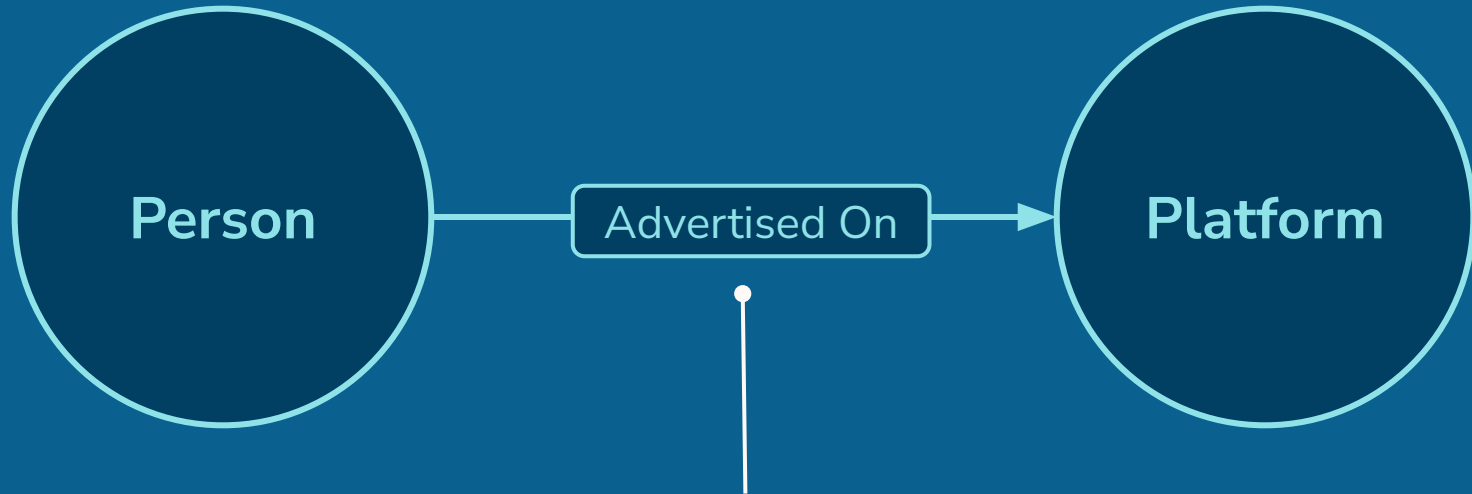
(Noun)-[Verb]->(Noun)



...and knowing when to break it



...and knowing when to break it

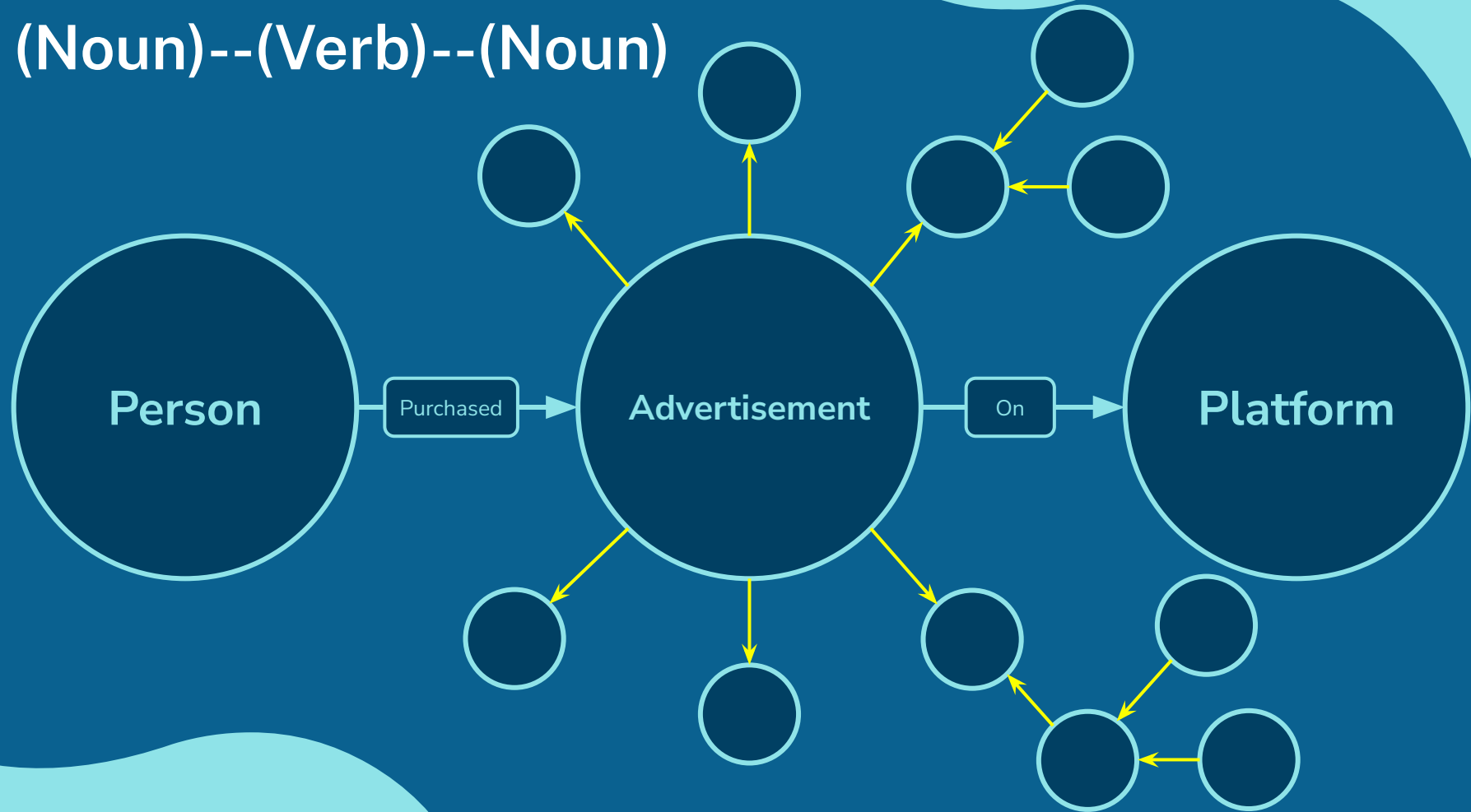


You may need to
build out a network
around this fact

...and knowing when to break it



(Noun)--(Verb)--(Noun)



Properties

Person

name: Martin
lives: UK
Last_seen: 2024-06-01

Person

name: Alex
lives: Germany
purchases: 5



Answering questions



Which is the best Matrix film?



Have a go!

Use the **rating** property
on the **RATED**
relationship to find:

1. The movie with the most ratings.

```
MATCH (m:Movie)<-[:?????]- (u:User)
RETURN m.title, count(?.?????) AS ratings
ORDER BY ratings DESC
```
2. The top 10 movies based on average rating.

```
MATCH (m:Movie)<-[:?????]- (u:User)
RETURN m.title, avg(?.?????) AS avgRating
LIMIT 10
ORDER BY avgRating DESC
```

Kevin Bacon <-> Al Pacino



Have a go!

Use the **shortestPath** function to find:

1. The connection between 2 Movie nodes.

```
MATCH (m1:Movie {title:"???????"})
MATCH (m2:Movie {title:"???????"})
MATCH p = shortestPath((m1)-[:ACTED_IN*]-(m2))
RETURN p
```

2. The connection between a random User and Kevin Bacon

```
MATCH (p:Person {title:"Kevin Bacon"})
MATCH (?:User {name:"Angela Thompson"})
MATCH p = ?????((?)-[*]-(?))
RETURN p
```



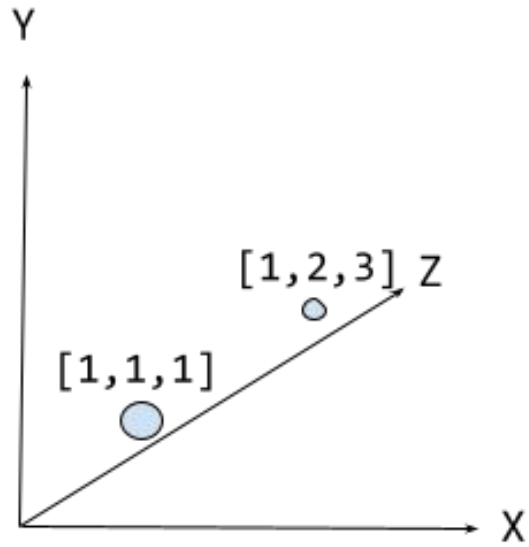

Vector indexes



Apple - Technology or Fruit



Vectors

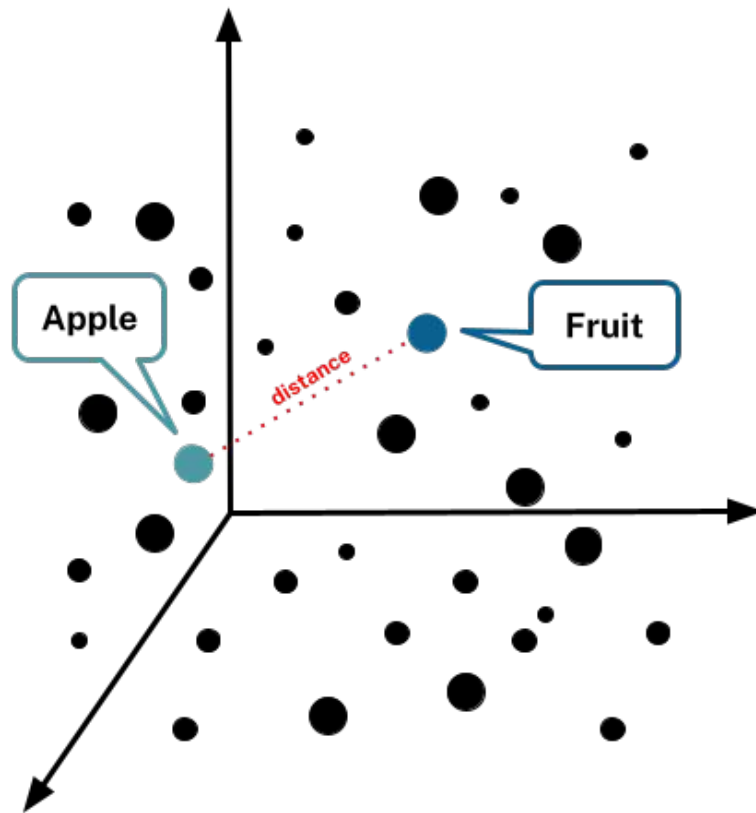


Embeddings

“apple”

```
[0.0077788467, -0.02306925, -0.007360777, -0.027743412,  
-0.0045747845, 0.01289164, -0.021863015, -0.008587573,  
0.01892967, -0.029854324, -0.0027962727, 0.020108491,  
-0.004530236, 0.009129008, ...]
```

Vector search



Movie plots

https://data.neo4j.com/rec-embed/plot_load.cypher

```
LOAD CSV WITH HEADERS
FROM 'https://data.neo4j.com/rec-embed/movie-plot-embeddings-1k.csv'
AS row
MATCH (m:Movie {movieId: row.movieId})
CALL db.create.setNodeVectorProperty(m, 'plotEmbedding',
apoc.convert.fromJsonList(row.embedding))
```

Vector index

https://data.neo4j.com/rec-embed/plot_create_index.cypher

```
CREATE VECTOR INDEX moviePlots IF NOT EXISTS
FOR (m:Movie)
ON m.plotEmbedding
OPTIONS {indexConfig: {
  `vector.dimensions`: 1536,
  `vector.similarity_function`: 'cosine'
}}
```

Query vector

https://data.neo4j.com/rec-embed/plot_query.cypher

```
MATCH (m:Movie {title: 'Toy Story'})
```

```
CALL db.index.vector.queryNodes('moviePlots', 6,  
m.plotEmbedding)
```

```
YIELD node, score
```

```
RETURN node.title, node.plot, score
```


Movie plots

https://data.neo4j.com/rec-embed/plot_load.cypher

https://data.neo4j.com/rec-embed/plot_create_index.cypher

https://data.neo4j.com/rec-embed/plot_query.cypher

Learn more with



- **Completely Free**
- **Hands-on Courses**
teaching Neo4j Fundamentals, Cypher, Drivers and Graph Data Science
- **Curated Learning Paths** catering for everyone from beginners to experts
- **Free Certifications**

graphacademy.neo4j.com

